

ClipQuest

Iteration overview for the 36-hour challenge

Project: ClipQuest

Repository: <https://github.com/404-Waifu-Not-Found/HHCC2026>

Live product: <https://clipquest.ccwu.cc>

Submission summary

This document describes the work completed during the event and the difference between the pre-event foundation and the post-event iteration.

Pre-event foundation

Before the event, the project had only a short written README describing the initial product idea. There was no implemented product flow, completed frontend, quiz engine, persistence layer, browser extension, native client flow, or production deployment to evaluate.

The event iteration turned that written concept into the codebase and working product submitted here.

What changed during the event

The team built the full product in this project. The work was iterative: each stage addressed a concrete product risk discovered in the previous stage.

Iteration	Problem or goal	Improvement delivered
Product foundation	A written idea needed a usable learning flow.	Built the web app, authentication, URL import, lesson preview, quiz flow, feedback, completion state, and Library.
Evidence-grounded generation	AI questions needed reliable lesson context.	Added caption-only public YouTube acquisition, normalization, structured DeepSeek generation, strict schemas, and explicit caption failure states.
Progressive learning	Waiting for a complete	Added question-first progressive generation so the first

	quiz felt slow.	validated question opens the attempt while the rest continue in bounded batches.
Correctness and recovery	A malformed question could break an attempt.	Added ordinal validation, duplicate checks, repair of the first missing question, idempotent imports, cancellation, timeouts, and safe recovery.
Mastery loop	A result alone did not create repeatable learning.	Added Library history, color-coded mastery, review state, progress bars, due reviews, saved lessons, and practice presentation.
Cross-platform access	The experience needed to work beyond one browser screen.	Added a Chrome extension bridge plus native iOS and Android paths sharing the local quiz engine and privacy boundary.
Product polish	The first UI needed stronger hierarchy and feedback.	Redesigned cards, hover/theme states, progress treatment, streaming feedback, empty/error states, responsive layouts, accessibility, and reduced motion.
Release readiness	A working feature needed repeatable delivery.	Added contracts, migrations, API routes, tests, asset verification, extension packaging, Wrangler deployment, health checks, and release docs.

Pre-event versus post-event

Area	Pre-event	Post-event
Product state	Short README only	Deployed web product with browser extension and native implementation paths
Input	Product idea	Public YouTube URL validation, metadata preview, and caption acquisition
AI	No implemented AI flow	Learner-owned DeepSeek key used locally for structured question generation
Learning	No quiz experience	Multiple-choice, true/false, and short-answer quizzes with immediate feedback
Progress	No persistence or mastery model	Authenticated attempts, ordered storage, mastery ranks, review state, and Library

Reliability	No validation or recovery	Contract validation, bounded retries, repair, cancellation, timeouts, and idempotency
UX	No interface	Responsive UI with light/dark themes, motion, accessibility, and clear states
Platforms	No working client	Web, Chrome extension bridge, iOS path, and Android path sharing core logic
Delivery	No deployment	Cloudflare Worker and static assets deployed at clipquest.ccwu.cc
Verification	No test suite	Automated contract, API, app, engine, extension, UI, build, and asset checks

Improvement logic and scope

1. **Ground the content.** Use public captions rather than invented lesson content. Unsupported or captionless sources fail explicitly.
2. **Protect correctness.** Validate every question before it becomes learner-visible or persistent. Accepted questions are never silently replaced.
3. **Reduce waiting.** Progressive generation makes question 1 available quickly while preserving ordered completion.
4. **Make progress meaningful.** Results become mastery, review, and Library state.
5. **Make the product usable.** Responsive layouts, theme parity, accessible controls, visible states, and motion-safe feedback are part of the feature.
6. **Ship with evidence.** Automated checks and production deployment are included; unfinished features remain clearly gated.

The current release intentionally hides the Workplace AI chat navigation for all users while that feature remains in development. The route and source remain behind a release gate for future work.

Team decisions and responsibilities

- Core product concept and UX flow
- Feature scope and prioritization
- Project architecture and file structure
- Caption-only privacy boundary

- Key algorithms and validation/recovery logic
- Mastery and review model
- Testing strategy and CI/release verification
- Visual direction, interaction design, and accessibility expectations

AI usage disclosure

Usage cost

\$240 in API-token usage across all tools

Tools used

- GitHub Copilot
- Claude Code CLI

All implementation code for the event iteration is written in this project repository.

Team decisions and concepts

- Project architecture and file structure
- Core product concept and UX flow
- Feature scope and design decisions
- Key algorithms and logic design
- Testing strategy and CI pipeline setup

AI as an auxiliary tool

AI was used only for boilerplate and repetitive code, syntax lookup and autocomplete, refactoring and code formatting, documentation drafting, and debugging assistance.

All key decisions, concepts, and creative direction were developed entirely by the team. AI served as an auxiliary coding tool only.

Recommended judge submission

1. **Project name:** ClipQuest

2. **Iteration code:** <https://github.com/404-Waifu-Not-Found/HHCC2026>
3. **Iteration overview:** [ITERATION-OVERVIEW.pdf](#)
4. **Presentation/demo:** <https://clipquest.ccwu.cc>
5. **AI disclosure:** Include the AI usage disclosure section above

The repository contains the complete event implementation. The README and documentation provide additional architecture, privacy, release, and QA context.